

Sistemas Operativos

Shell y syscalls

Marcelo Arroyo

Dpto de Computación - FCEFQyN
Universidad Nacional de Río Cuarto

2024



Contenidos

1. Introducción
2. Multiprogramación y reubicabilidad
3. Segmentación
4. Paginado
5. Caché y TLBs
6. Memoria virtual
7. Reemplazo de páginas
8. Gestión de memoria en XV6



Introducción



Objetivos

- Uso eficiente
- Asignación de bloques de memoria requeridos por los procesos y el kernel
- Protección: Un proceso debe acceder a áreas de memoria permitidas y en el modo permitido
- Gestión de la jerarquía de memoria (caché, RAM, buffers de E/S, ...)

Memoria virtual

Uso de dispositivos de almacenamiento secundario como extensión de la RAM para poder ejecutar procesos que requieran más RAM de la existente



Manejo del heap

El kernel deberá mantener un heap (pool) de memoria para poder asignar y liberar bloques antes las demandas de los procesos y el mismo kernel.

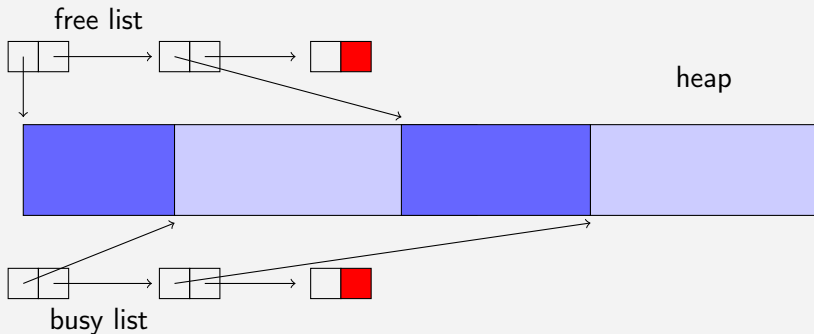
Técnicas

El heap requiere mantener conjuntos de bloques libres y asignados (ocupados)

- Bloques de tamaño fijo
- Bloques de tamaño variable

Asignación de memoria (cont.)

Bloques de tamaño variable



Asignación de memoria (cont.)

Problemas con bloques de tamaño variable

- *Fragmentación externa*: Muchos bloques libres pequeños
- *Listas con descriptores*: Con tamaño de cada bloque



Asignación de memoria (cont.)

Problemas con bloques de tamaño variable

- *Fragmentación externa*: Muchos bloques libres pequeños
- *Listas con descriptores*: Con tamaño de cada bloque

Soluciones a la fragmentación

- Compactación (movimiento y fusión) de bloques libres
- Semi-compactación: Fusión de bloques libres adyacentes



Asignación de memoria (cont.)

Problemas con bloques de tamaño variable

- *Fragmentación externa*: Muchos bloques libres pequeños
- *Listas con descriptores*: Con tamaño de cada bloque

Soluciones a la fragmentación

- Compactación (movimiento y fusión) de bloques libres
- Semi-compactación: Fusión de bloques libres adyacentes

Estrategias de asignación

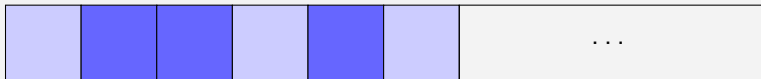
- Primer ajuste (first fit)
- Mejor ajuste (best fit)
- Siguiendo ajuste (memorización del último bloque asignado)



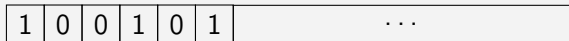
Asignación de memoria (cont.)

Bloques de igual tamaño

Heap



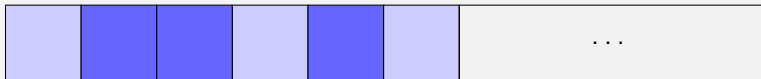
Bitmap



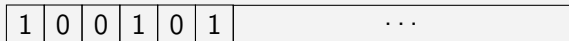
Asignación de memoria (cont.)

Bloques de igual tamaño

Heap



Bitmap



Bloques de igual tamaño

- **Ventaja:** simplicidad
- **Desventaja:** fragmentación interna
- **¿Tamaño de bloque?**

Asignación de memoria (cont.)

Sistema compañero (buddy system)

- Conjuntos de bloques libres de tamaño entre 2^l y 2^h (Kb)
 1. Inicialmente hay bloques libres de tamaño 2^h



Asignación de memoria (cont.)

Sistema compañero (buddy system)

- Conjuntos de bloques libres de tamaño entre 2^l y 2^h (Kb)
 1. Inicialmente hay bloques libres de tamaño 2^h
 2. En una solicitud de tamaño m :
 - 2.1 Buscar un bloque libre de mejor ajuste ($b_i : m \geq 2^i$)
 - 2.2 Si existe, asignarlo
 - 2.3 Sino:
 - 2.4 Dividir a la mitad un bloque de tamaño 2^{i+1}
 - 2.5 Si se alcanzó tamaño de mejor ajuste, asignarlo
 - 2.6 Sino, repetir con $2^{i+2}, \dots$

Ejemplo con $l = 16(2^l = 64Kb)$, $h = 19(2^h = 512Kb)$

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	512K							

Asignación de memoria (cont.)

Sistema compañero (buddy system)

- Conjuntos de bloques libres de tamaño entre 2^l y 2^h (Kb)
 1. Inicialmente hay bloques libres de tamaño 2^h
 2. En una solicitud de tamaño m :
 - 2.1 Buscar un bloque libre de mejor ajuste ($b_i : m \geq 2^i$)
 - 2.2 Si existe, asignarlo
 - 2.3 Sino:
 - 2.4 Dividir a la mitad un bloque de tamaño 2^{i+1}
 - 2.5 Si se alcanzó tamaño de mejor ajuste, asignarlo
 - 2.6 Sino, repetir con $2^{i+2}, \dots$

Ejemplo con $l = 16(2^l = 64Kb)$, $h = 19(2^h = 512Kb)$

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	512K							
$P_0 : 34K$	P_0	64K	128K		256K			

Asignación de memoria (cont.)

Sistema compañero (buddy system)

- Conjuntos de bloques libres de tamaño entre 2^l y 2^h (Kb)
 1. Inicialmente hay bloques libres de tamaño 2^h
 2. En una solicitud de tamaño m :
 - 2.1 Buscar un bloque libre de mejor ajuste ($b_i : m \geq 2^i$)
 - 2.2 Si existe, asignarlo
 - 2.3 Sino:
 - 2.4 Dividir a la mitad un bloque de tamaño 2^{i+1}
 - 2.5 Si se alcanzó tamaño de mejor ajuste, asignarlo
 - 2.6 Sino, repetir con $2^{i+2}, \dots$

Ejemplo con $l = 16(2^l = 64Kb)$, $h = 19(2^h = 512Kb)$

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	512K							
$P_0 : 34K$	P_0	64K	128K		256K			
$P_1 : 95K$	P_0	64K	P_1		256K			



Asignación de memoria (cont.)

Sistema compañero (buddy system)

- Conjuntos de bloques libres de tamaño entre 2^l y 2^h (Kb)
 1. Inicialmente hay bloques libres de tamaño 2^h
 2. En una solicitud de tamaño m :
 - 2.1 Buscar un bloque libre de mejor ajuste ($b_i : m \geq 2^i$)
 - 2.2 Si existe, asignarlo
 - 2.3 Sino:
 - 2.4 Dividir a la mitad un bloque de tamaño 2^{i+1}
 - 2.5 Si se alcanzó tamaño de mejor ajuste, asignarlo
 - 2.6 Sino, repetir con $2^{i+2}, \dots$

Ejemplo con $l = 16(2^l = 64Kb)$, $h = 19(2^h = 512Kb)$

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	512K							
$P_0 : 34K$	P_0	64K	128K		256K			
$P_1 : 95K$	P_0	64K	P_1		256K			
$P_2 : 55K$	P_0	P_2	P_1		256K			



Asignación de memoria (cont.)

Sistema compañero (buddy system)

- Conjuntos de bloques libres de tamaño entre 2^l y 2^h (Kb)
 1. Inicialmente hay bloques libres de tamaño 2^h
 2. En una solicitud de tamaño m :
 - 2.1 Buscar un bloque libre de mejor ajuste ($b_i : m \geq 2^i$)
 - 2.2 Si existe, asignarlo
 - 2.3 Sino:
 - 2.4 Dividir a la mitad un bloque de tamaño 2^{i+1}
 - 2.5 Si se alcanzó tamaño de mejor ajuste, asignarlo
 - 2.6 Sino, repetir con $2^{i+2}, \dots$

Ejemplo con $l = 16(2^l = 64Kb)$, $h = 19(2^h = 512Kb)$

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	512K							
$P_0 : 34K$	P_0	64K	128K		256K			
$P_1 : 95K$	P_0	64K	P_1		256K			
$P_2 : 55K$	P_0	P_2	P_1		256K			
$P_3 : 90K$	P_0	P_2	P_1		P_3		128K	



Sistema compañero (buddy system), cont.

- En una liberación de un bloque:
 1. Marcar el bloque como libre
 2. Si hay un vecino libre:
 - 2.1 Combinarlos y volver al paso 2

Sistema compañero (buddy system), cont.

- En una liberación de un bloque:
 1. Marcar el bloque como libre
 2. Si hay un vecino libre:
 - 2.1 Combinarlos y volver al paso 2

Continuación del ejemplo...

Req.	64K	64K	64K	64K	64K	64K	64K	64K

Asignación de memoria (cont.)

Sistema compañero (buddy system), cont.

- En una liberación de un bloque:
 1. Marcar el bloque como libre
 2. Si hay un vecino libre:
 - 2.1 Combinarlos y volver al paso 2

Continuación del ejemplo...

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	P_0	P_2	P_1		P_3		128K	

Asignación de memoria (cont.)

Sistema compañero (buddy system), cont.

- En una liberación de un bloque:
 1. Marcar el bloque como libre
 2. Si hay un vecino libre:
 - 2.1 Combinarlos y volver al paso 2

Continuación del ejemplo...

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	P_0	P_2	P_1		P_3		128K	
$freeP_2$	P_0	64K	P_1		P_3		128K	

Asignación de memoria (cont.)

Sistema compañero (buddy system), cont.

- En una liberación de un bloque:
 1. Marcar el bloque como libre
 2. Si hay un vecino libre:
 - 2.1 Combinarlos y volver al paso 2

Continuación del ejemplo...

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	P_0	P_2	P_1		P_3		128K	
$freeP_2$	P_0	64K	P_1		P_3		128K	
$freeP_0$	128K		P_1		P_3		128K	

Asignación de memoria (cont.)

Sistema compañero (buddy system), cont.

- En una liberación de un bloque:
 1. Marcar el bloque como libre
 2. Si hay un vecino libre:
 - 2.1 Combinarlos y volver al paso 2

Continuación del ejemplo...

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	P_0	P_2	P_1		P_3		128K	
$freeP_2$	P_0	64K	P_1		P_3		128K	
$freeP_0$	128K		P_1		P_3		128K	
$freeP_1$	256K				P_3		128K	

Asignación de memoria (cont.)

Sistema compañero (buddy system), cont.

- En una liberación de un bloque:
 1. Marcar el bloque como libre
 2. Si hay un vecino libre:
 - 2.1 Combinarlos y volver al paso 2

Continuación del ejemplo...

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	P_0	P_2	P_1		P_3		128K	
$freeP_2$	P_0	64K	P_1		P_3		128K	
$freeP_0$	128K		P_1		P_3		128K	
$freeP_1$	256K				P_3		128K	
$freeP_3$	512K							

Asignación de memoria (cont.)

Asignador de potencias de 2

- Listas de bloques de tamaños $= 2^k$ ($2^l \leq k \leq 2^h$)
- Listas libres por separado
- No se combinan bloques
- Dado un requerimiento de bloque de tamaño m :
 1. Se localiza la menor lista i tal que $2^i \geq m$ y tenga un bloque libre
 2. Si no existe ninguna, el requerimiento falla

Interface del manejador del heap

- Alto nivel: ***`void* malloc(size_t size), void free(void* ptr)`***
- Bajo nivel: ***`void* alloc_frame(), void free_frame(void* ptr)`***



Multiprogramación y reubicabilidad



Ejecución de código independiente de su posición

Múltiples programas cargados en memoria

- Los programas de usuario comúnmente se compilan para ser ejecutados a partir de cierta dirección de memoria: Cada función o variable global tiene una dirección asignada
- Problema: Varios programas esperan ser cargados desde la misma dirección base
- Posibles soluciones:
 1. Compilación a ***P**osition independent code (PIC)*
 2. ***U**so de Memory Management Unit (MMU)*



Código independiente de la posición (PIC)

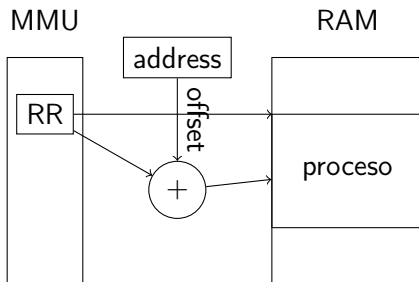
1. Uso de un *registro de link/load* que se inicializa por el SO con su dirección base de carga
2. Las instrucciones usan *direccionamiento relativo* a ese registro u otro (ej: al *program counter*)
3. En CPUs sin esta maquinaria, el linker/loader deberá *recalcular* las direcciones de los operandos de las instrucciones en tiempo de carga, antes de iniciar su ejecución
4. En CPUs con MMUs, las direcciones de un programa son *lógicas*. La MMU *traduce* las direcciones lógicas en reales (*físicas*)



Memory Management Unit (MMU)

Registros base

- Uso de un *registro de reubicabilidad RR*
- Cada dirección de memoria se interpreta como un *offset*
- La MMU implementa la función $phys_addr = RR + offset$



Segmentación



Objetivos

- Reubicabilidad
- Protección: Evitar accesos a memoria no permitidos

Objetivos

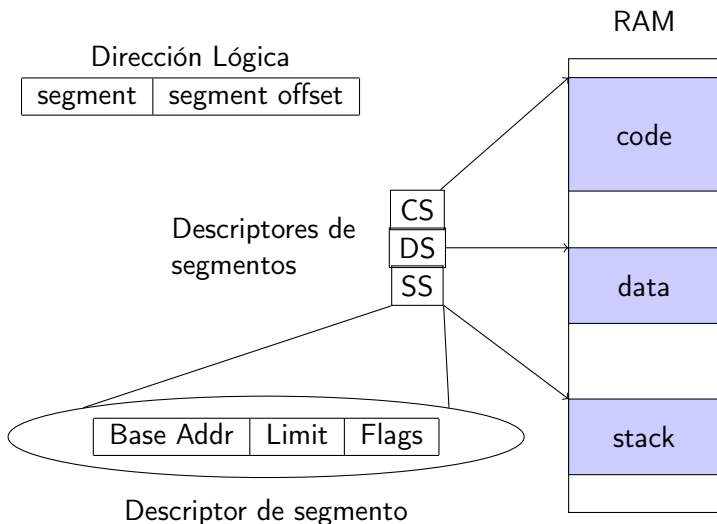
- Reubicabilidad
- Protección: Evitar accesos a memoria no permitidos

Segmento: *Unidad lógica de un proceso*

1. Segmento de código (instrucciones del programa)
2. Segmento de datos (variables globales)
3. Stack
 - Control de llamadas a subrutinas (y retornos)
 - Variables locales (automáticas)
 - Valores temporarios (cálculos de expresiones)
4. Heap (variables creadas dinámicamente)

Segmentación

Un conjunto de descriptores de segmentos por proceso

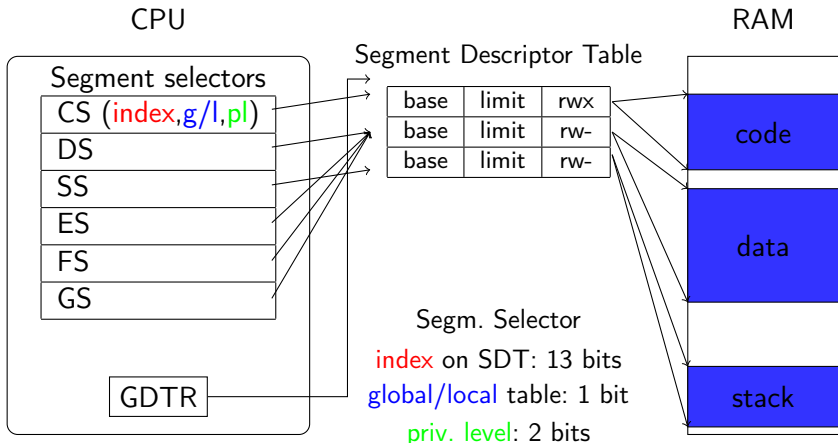


Segmentación en IA32

Logical address

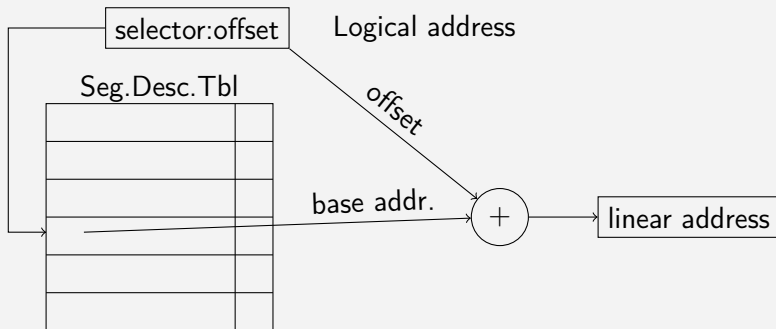
`segment:offset`

(Ej: `mov EAX, DS:0xFF`)



Segmentación en IA32 (cont.)

Direcccionamiento



Protección: La CPU genera una Segmentation fault

- En un acceso fuera de los límites de un segmento
- En un acceso a un segmento sin los permisos necesarios



Paginado

Características

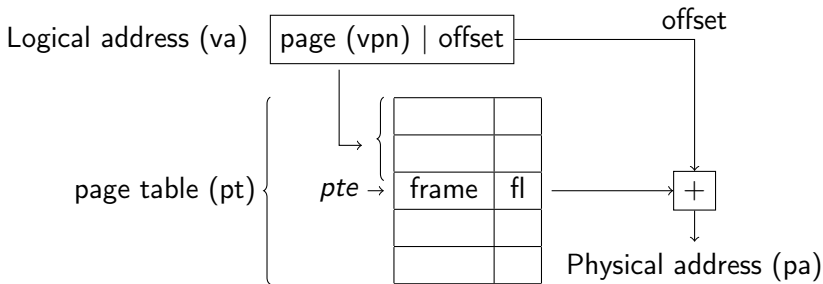
- Memoria particionada en bloques de tamaño fijo (páginas)
- Tamaños de páginas comúnmente usadas: 4KB o 4MB
- Ventaja sobre segmentación:
 1. Gestión simple del heap
 2. Particionado de memoria más fina
 3. Permite implementar técnicas de *virtual memory*

Implementación: Tablas de páginas

- Virtual/logical address (va): Par (*vpn, offset*)
- Tabla de páginas: *pt: array[0..N-1] of pte*
- Page table entry (pte): Par (*ppn, flags*)

Paginado: Esquema

- Tabla de páginas: $pt : logical_frame \rightarrow physical_frame$
- Mapping: $pa = pt[va.vpn].ppn + va.offset^1$



¹ vpn/ppn : virtual/physical page number.

Características

- Páginas de 4KB o 4MB
- Páginas de 4KB: Árbol de dos niveles
 1. Tablas de páginas (nodos) de 1024 entradas
 2. Directorio de páginas (raíz, apuntado por CR3)
- Uso con segmentación:
paging : linear address $\rightarrow$ physical address

Detalles

- Dirección lógica (4kb):

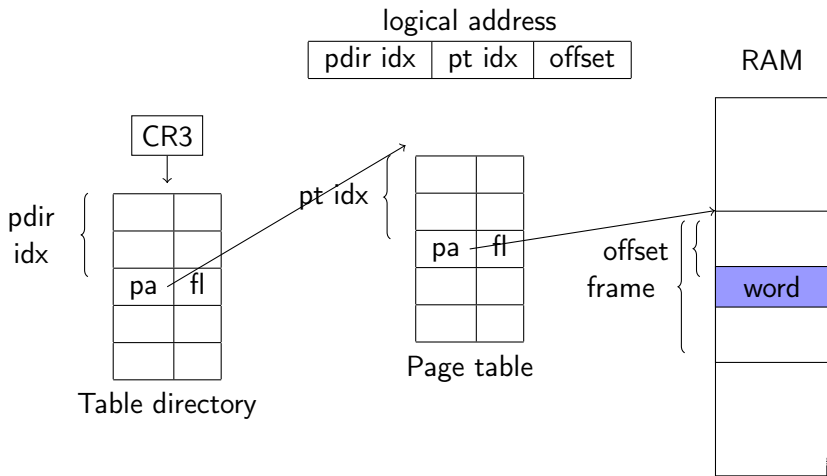
pgdir idx (10 bits)	pgtbl idx (10 bits)	offset (12 bits)
---------------------	---------------------	------------------

- Entrada en tabla de página:

physical address (20 bits)	flags
----------------------------	-------

Flags: **U**ser, **P**resent, **A**ccessed, **D**irty, ...

Paginado en IA32 (cont.)



Paginado en riscv-64

- El registro Supervisor Address Translation and Protection (SATP) apunta a la tabla de páginas (raíz)
- Páginas de 4KB.
- El *riscv Sv39* las *direcciones virtuales* son de 39 bits

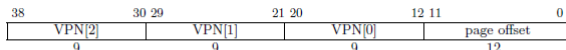


Figure 4.16: Sv39 virtual address.

VPN: Virtual Page Number: *index* en una tabla de páginas.

- Tabla de páginas: Árbol de 3 niveles
- Cada nodo del árbol es una tabla de 512 entradas (*pte*) de 64bits c/u

Paginado en riscv-64

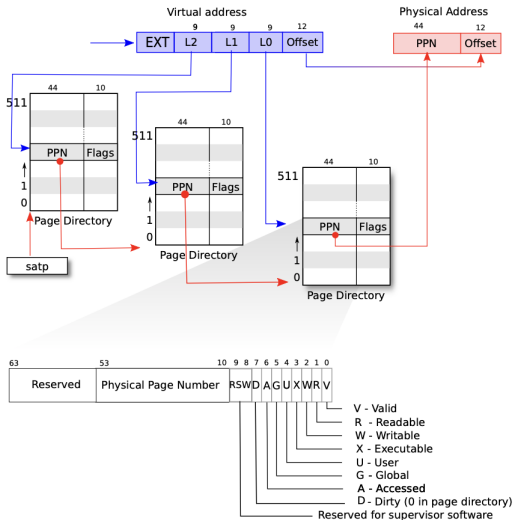


Figure 3.2: RISC-V address translation details.

Caché y TLBs



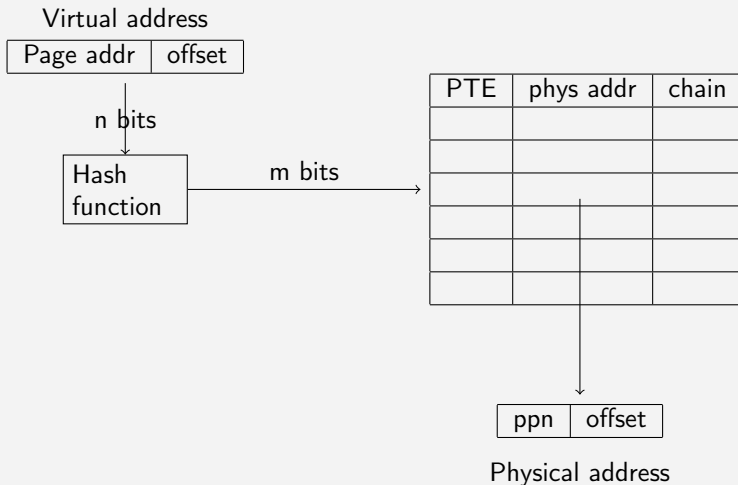
Traslation Lookaside Buffer (TLB)

- Cada referencia a memoria requiere acceso a tablas de páginas
- Para evitar tantos accesos a memoria se requiere TLBs
- Es una caché de alta velocidad de entradas de tablas de páginas (PTEs)
- Si una PTE no se encuentra en la TLB (*cache miss*) se carga desde la memoria.
- Generalmente se usa un algoritmo de reemplazo *least recently used* (LRU)



Traslation Lookaside Buffer

Implementación: memoria asociativa



Memoria virtual



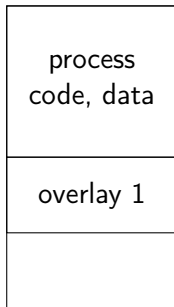
Objetivos y aplicaciones

- Extender lógicamente la RAM en medios de almacenamiento (discos)
- Permite que un proceso vea más memoria que la físicamente disponible.
- Permite ejecutar programas *parcialmente cargados en memoria*
- Carga de partes de procesos bajo demanda (mmap)
- Bloques no referenciados nunca se cargarán
- Permite ejecutar más procesos de los que podrían albergarse en memoria
- Mejores tiempos de respuesta (transferencias de E/S cortas)
- Copia de procesos (en `fork()`) bajo demanda (*copy-on-write*)
- Otros: *bibliotecas compartidas ...*



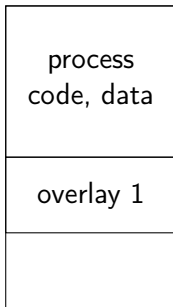
Overlays: módulos cargables dinámicamente

- Carga/descarga de módulos de un programa bajo demanda
- Util en programas con operaciones secuenciales. Ejemplo:
`codegen(parse(src))`



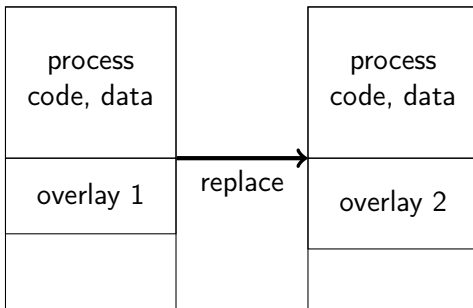
Overlays: módulos cargables dinámicamente

- Carga/descarga de módulos de un programa bajo demanda
- Util en programas con operaciones secuenciales. Ejemplo:
`codegen(parse(src))`



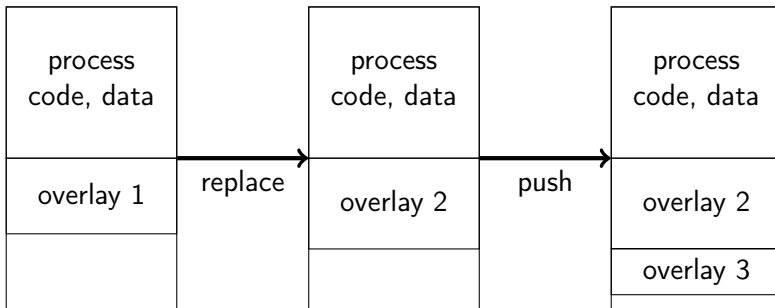
Overlays: módulos cargables dinámicamente

- Carga/descarga de módulos de un programa bajo demanda
- Util en programas con operaciones secuenciales. Ejemplo:
`codegen(parse(src))`



Overlays: módulos cargables dinámicamente

- Carga/descarga de módulos de un programa bajo demanda
- Util en programas con operaciones secuenciales. Ejemplo: `codegen(parse(src))`



Mecanismos (permisos y flags)

Page table entry

physical address	flags
------------------	-------

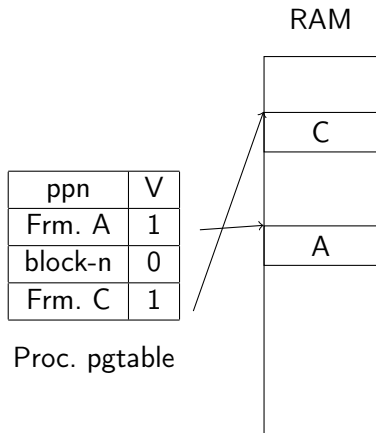
- Flag *V/P* (*valid/present*) en cada entrada en tablas de páginas. Hardware causa **page fault** si bit *V* es cero en un acceso a la *pte*
- Flag *A* (*accessed*): MMU setea este bit en cada acceso
- Flag *D* (*dirty*): MMU setea este bit en cada escritura
- *Permisos de acceso (R,W,X)*: Generan *page faults* en accesos indebidos

Memoria virtual: Swapping

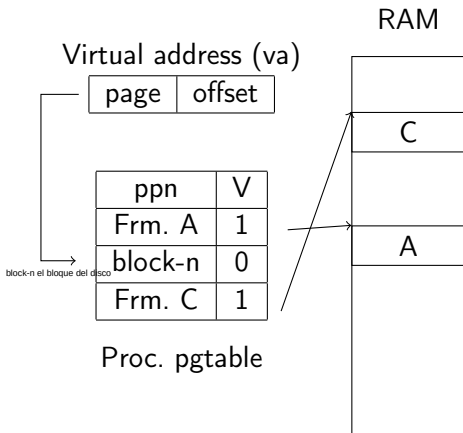
```
// free_list points to first page of free pages list
uint64 allocpage()
{
    if (free_list) {
        uint64 p = free_list;
        free_list = *free_list.next;
        return p;
    } else {
        // no free page. Choose one and swap out
        struct PTE * pte = select_victim();
        uint64 pa = pte.ppn * PG_SIZE;
        disk_block_no = write_to_disk(pte.ppn);
        clear_page(*pa);
        pte->v = 0;
        pte->ppn = disk_block_no;
        return pa;
    }
}
```



Memoria virtual: Swapp-in

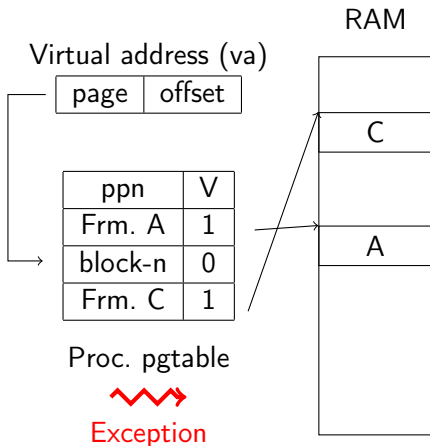


Memoria virtual: Swapp-in



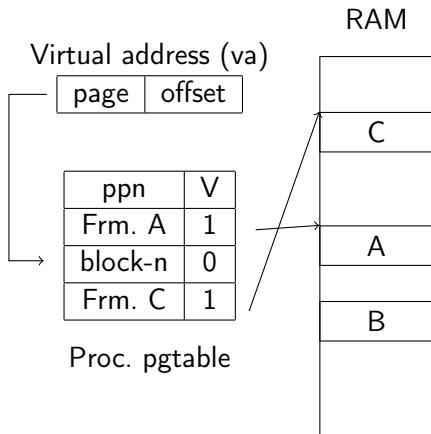
1. Instruction uses va

Memoria virtual: Swapp-in



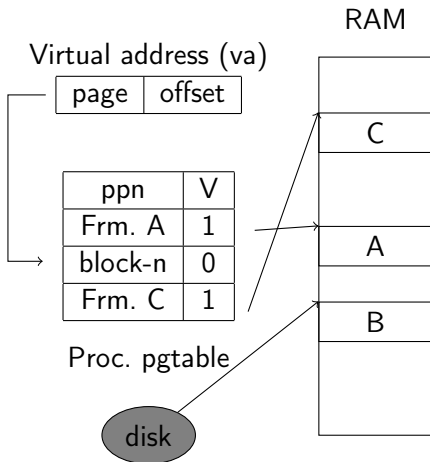
1. Instruction uses va
2. **Page fault** (`pte.valid == 0`)
3. PageFaultHandler:

Memoria virtual: Swapp-in



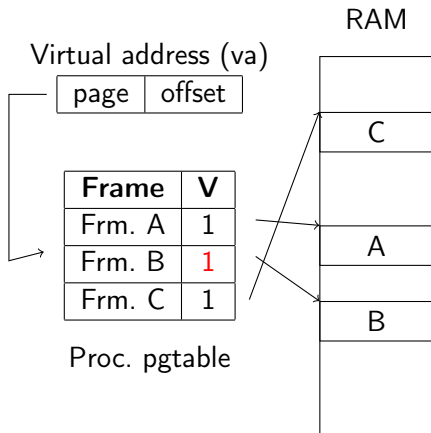
1. Instruction uses va
2. **Page fault** (`pte.valid == 0`)
3. PageFaultHandler:
3.1 `pa = allocpage()`

Memoria virtual: Swapp-in



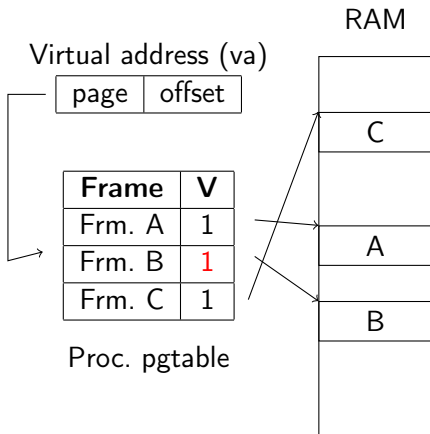
1. Instruction uses va
2. **Page fault** (`pte.valid == 0`)
3. PageFaultHandler:
 - 3.1 `pa = allocpage()`
 - 3.2 `load_block(n, pa)`

Memoria virtual: Swapp-in



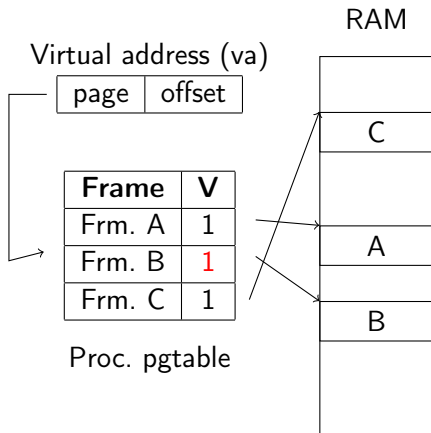
1. Instruction uses va
2. **Page fault** (`pte.valid == 0`)
3. PageFaultHandler:
 - 3.1 `pa = allocpage()`
 - 3.2 `load_block(n, pa)`
 - 3.3 `map_page(va, pa)`

Memoria virtual: Swapp-in



1. Instruction uses va
2. **Page fault** (`pte.valid == 0`)
3. PageFaultHandler:
 - 3.1 `pa = allocpage()`
 - 3.2 `load_block(n, pa)`
 - 3.3 `map_page(va, pa)`
 - 3.4 `sret`

Memoria virtual: Swapp-in



1. Instruction uses va
2. **Page fault** (`pte.valid == 0`)
3. PageFaultHandler:
 - 3.1 `pa = allocpage()`
 - 3.2 `load_block(n, pa)`
 - 3.3 `map_page(va, pa)`
 - 3.4 `sret`
4. CPU Re-exec instruction

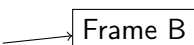
Una vez retornado el manejo de la excepcion se vuelve a la instruccion que causo el PageFault Porque lo guarda en el SEPC una vez que se hace el trap del PageFault, pudiendo volver a la instruccion que la causo, y hacer su reejecucion. Pero mas adelante cuando hagamos llamadas al sistema (las cuales hacen interrupciones al sistema), debemos saltar la instruccion a la siguiente.



Memoria virtual: Copy on write

1. Proceso padre

Page	W
A	0
B	1
C	1

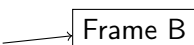


pagetable proceso padre

Memoria virtual: Copy on write

1. Proceso padre
2. `fork()`

Page	W
A	0
B	1
C	1



pagetable proceso padre

Memoria virtual: Copy on write

1. Proceso padre
2. `fork()`
 - 2.1 $W = 0$ a pte's

Page	W
A	0
B	0
C	0

→ Frame B

pagetable proceso padre

Memoria virtual: Copy on write

1. Proceso padre
2. `fork()`
 - 2.1 $W = 0$ a pte's
 - 2.2 Copiar pagetable

pagetable proceso hijo

Page	W
A	0
B	0
C	0

Page	W
A	0
B	0
C	0

Frame B

pagetable proceso padre

Memoria virtual: Copy on write

1. Proceso padre
2. **fork()**
 - 2.1 $W = 0$ a pte's
 - 2.2 Copiar pagetable
3. Proc. hijo intenta escribir en frame B
 - 3.1 **Page fault**
 - 3.2 PageFaultHandler:

pagetable proceso hijo

Page	W
A	0
B	0
C	0

Page	W
A	0
B	0
C	0

Frame B

pagetable proceso padre

Memoria virtual: Copy on write

1. Proceso padre
2. **fork()**
 - 2.1 $W = 0$ a pte's
 - 2.2 Copiar pagetable
3. Proc. hijo intenta escribir en frame B
 - 3.1 **Page fault**
 - 3.2 PageFaultHandler:
 - 3.2.1 Copiar frame

pagetable proceso hijo

Page	W
A	0
B	0
C	0

Page	W
A	0
B	0
C	0



pagetable proceso padre

Memoria virtual: Copy on write

1. Proceso padre
2. **fork()**
 - 2.1 $W = 0$ a pte's
 - 2.2 Copiar pagetable
3. Proc. hijo intenta escribir en frame B
 - 3.1 **Page fault**
 - 3.2 PageFaultHandler:
 - 3.2.1 Copiar frame
 - 3.2.2 Mapear frame en el hijo
 - 3.2.3 $W = 1$ en ambas tablas

pagetable proceso hijo

Page	W
A	0
B'	1
C	0

Frame B'

Page	W
A	0
B	1
C	0

Frame B

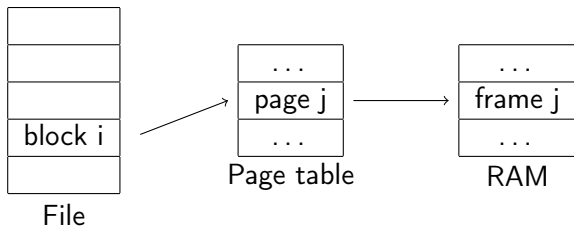
pagetable proceso padre

Mapeo de archivos en memoria

- I/O de archivos implícito para el programador
- Uso:
 1. `int fd = open(filepath, ...);`
 2. `char* ptr = mmap(..., size, flags, fd, ...);`
Mapea el archivo y retorna la dirección virtual del área de memoria mapeada (*no aún allocated*) para su contenido
 3. `char c = ptr[0];`
Esto causará un *page fault*. El OS cargará los bloques del disco a la página correspondiente (*on demand allocation*)
 4. `c = ptr[5];` no causará un *page fault*, su contenido ya se encuentra en memoria
 5. `munmap(ptr, size)`: Libera el mapping, posiblemente escribiendo a disco los bloques modificados en la memoria
- Mapeo *anónimo* (flag `MAP_ANON`), reserva espacio de direcciones sin asociar páginas a bloques de archivos



Mmap/munmap: Implementación



1. El archivo se divide lógicamente en n páginas
2. Se *reservan* n *page table entries* consecutivas con el bit $V=0$ con $PPN=file_block_number$
3. La carga se hace bajo demanda (on *page faults*)
4. En `munmap()`, se *escriben* las (*páginas*) modificadas (bit $D=1$)

Memoria Virtual (cont.)

Memoria compartida:

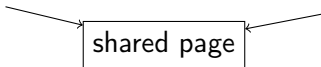
Tablas de páginas de diferentes procesos describen mismas páginas

Process A page table

...
phys. address
...

Process B page table

...
...
phys. address
...



Notar que las direcciones lógicas (virtuales) difieren!

Reemplazo de páginas



Reemplazo de páginas

Estos algoritmos pueden usarse en gestión de RAM (paginado) o de caché.

- Algoritmos justos (ej: *FIFO*)
- Algoritmo ideal de elección de la víctima:
 - Aquella que se referenciará más adelante en el futuro (eventualmente nunca)



Reemplazo de páginas

Estos algoritmos pueden usarse en gestión de RAM (paginado) o de caché.

- Algoritmos justos (ej: *FIFO*)
- Algoritmo ideal de elección de la víctima:
 - Aquella que se referenciará más adelante en el futuro (eventualmente nunca)
 - **Requiere predecir el futuro!!!**



Reemplazo de páginas

Estos algoritmos pueden usarse en gestión de RAM (paginado) o de caché.

- Algoritmos justos (ej: *FIFO*)
- Algoritmo ideal de elección de la víctima:
 - Aquella que se referenciará más adelante en el futuro (eventualmente nunca)
 - **Requiere predecir el futuro!!!**
 - Heurísticas que intentan *acercarse al algoritmo ideal*
 - ▶ Frecuencia o cantidad de accesos a cada página
 - ▶ Conjunto de trabajo (principios de localidad)



- **FIFO:** FirstIn-FirstOut. No ofrece buen rendimiento.
Anomalía de Belady: la tasa de faltas de página se incrementa al aumentar la capacidad de memoria (nro de páginas)



- **FIFO:** FirstIn-FirstOut. No ofrece buen rendimiento.
Anomalía de Belady: la tasa de faltas de página se incrementa al aumentar la capacidad de memoria (nro de páginas)
- **Second chance:** Extensión de FIFO
 1. Si la cabeza fue accedida (bit $A = 1$), se pone $A = 0$ y se la mueve a la cola
 2. Si no fue accedida, se selecciona como víctima
 3. **Reloj:** Variante que usa un puntero (*hand*) del último elemento examinado. Esto implementa una cola circular



No recientemente usada (NRU)

Periódicamente, apagar el bit *referenced* en cada pte

Ej: Bit *R* en x86 o el bit *A* en risc-v.

Ante una falta de página, la víctima a elegir se selecciona mediante el siguiente criterio:

1. $(A, D) = (0, 0)$: no accedida recientemente
2. $(A, D) = (0, 1)$: no accedida en el último período, escrita en el período anterior
3. $(A, D) = (1, 0)$: accedida recientemente
4. $(A, D) = (1, 1)$: accedida recientemente para escritura

El algoritmo selecciona una página (aleatoriamente) de la categoría más baja



- Least Recently Used (LRU):
 - Requiere llevar el tiempo del *último acceso* en cada página
 - Difícil implementación sin soporte de hardware

²R: *referenced* bit.

- **Least Recently Used (LRU):**
 - Requiere llevar el tiempo del *último acceso* en cada página
 - Difícil implementación sin soporte de hardware
- **No Frequently Used (NFU)**
 1. Asociamos un contador (*counter*) por cada página.
 2. Periódicamente a cada $counter_p + = R^2$
 3. En un page out, Se selecciona la de menor valor de *counter*
 4. Problema: *aging*. Contamos número de accesos, no cuándo ocurrieron
 5. Posible solución:
$$counter = (counter \gg 1) \mid (R \ll (countersize-1))$$

²R: *referenced* bit.

Working set ($ws(p)$)

Páginas referenciadas por el proceso p en el último período τ

1. Se asocia a cada página p el *tiempo del último acceso* ta
2. $ws(p)$ representado como una *cola circular*
3. Periódicamente (τ ticks) se recorre la cola. Sea la página p
 - 3.1 Si el bit $A = 1$: $ta_p = \tau$ y $A = 0$
 - 3.2 Si no, si $ta_p > \tau$, se elimina de $ws(p)$
Si $D = 1$ se planifica el *swap-out*
4. Se elige como víctima una página p con $ta_p > \tau$ y $D = 0$

- Estrategias:
 - **Global**: Se puede elegir cualquier página
 - **Local**: Desde el conjunto asignado al proceso
- Bloqueo de páginas (no elegibles como víctimas)
- Páginas compartidas (*shared memory*, *shared libraries*)
- **Sobrepaginación (thrashing)**: intercambio (swapping) excesivo
 - Alta frecuencia de faltas de página (*PFF*)
 - Soluciones: Incrementar memoria o eliminar procesos o mantener siempre un mínimo de páginas disponibles



Gestión de memoria en XV6



Asignador de bloques (allocation) físicos

- Asignador de bloques de tamaño uniforme (4Kb)
- Los bloques se enlazan en una *freelist* (ver `kalloc.c`)
- API: `char * kalloc(void)`, `void kfree(char *)`



Asignador de bloques (allocation) físicos

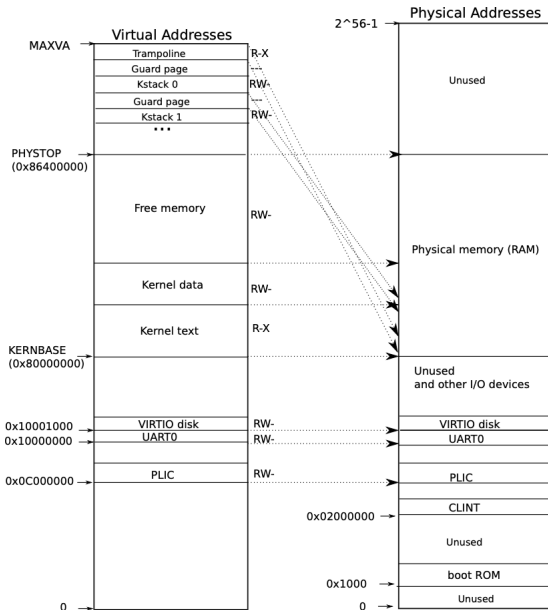
- Asignador de bloques de tamaño uniforme (4Kb)
- Los bloques se enlazan en una *freelist* (ver `kalloc.c`)
- API: `char * kalloc(void)`, `void kfree(char *)`

Mapa de memoria (kernel y procesos)

- El kernel tiene su propia tabla de páginas (mapea 2Gb-..., usado por `scheduler()`). Ver `kvmmake()`, en `vm.c`
- Para cada proceso se crea un nuevo *mapping* (`proc->pagetable`) donde:
 1. $[0, p - > sz]$, ($p - > pz < KERNBASE$): Espacio de direcciones virtuales para procesos
 2. $[KERNBASE, PHYSTOP]$: Espacio de direcciones virtuales del kernel



xv6: Mapeo de la memoria



xv6: Memoria virtual

vm.c: Módulo de *memoria virtual*

- `mappages(&pagetable, va, count, pa)`
- `walk(): virtual_address  $\rightarrow$  &pte`



xv6: Memoria virtual

vm.c: Módulo de *memoria virtual*

- `mappages(&pagetable, va, count, pa)`
- `walk(): virtual_address  $\rightarrow$  &pte`

Cambios de contexto

- El mapeo descripto no requiere hacer cambios de mapa de memoria al ocurrir una interrupción.
- Al ocurrir una interrupción/excepción en un proceso:
 1. La CPU pasa a modo supervisor
 2. Se salta a trampoline (código del kernel) mapeado por cada proceso
 3. trampoline salva los registros en el trapframe y cambia la tabla del kernel
 4. Antes de retornar (`userret`), `satp` apunta a la tabla de páginas del proceso

